

s15 Agent Team 架构与实现复盘笔记

围绕 Lead Loop、teammate、MessageBus、check_inbox、subagent 与 system prompt cache 的本次对话总结

核心结论

【重点】你这次的主线，是把前面 s10-s14 的单 Agent 主循环，继续推进到 s15 的 Agent Team：Lead Agent 负责调度，teammate 后台执行，MessageBus 负责通信，check_inbox 负责把异步结果重新交给 Lead。

本 PDF 按“架构理解 -> subagent 定位 -> 代码审查 -> 实现方案 -> cache hit 原因 -> 测试清单”的顺序整理。

0. 本次对话时间线

阶段	你问的问题	整理后的结论
1	看图后，这个 s15 Agent Team 架构是否合理？	大方向正确：Lead 保留主控权，teammate 独立执行，MessageBus 解耦通信，cron/background/inbox 作为外部事件源。
2	subagent 还需要吗？	需要，但不再作为高层工具暴露；它应降级为 teammate 内部的一次性执行内核 run_subagent_once。
3	按照教程简化方式写的代码对吗？	架构骨架正确，但存在工具 schema 未暴露、path/file_path 不匹配、SUB_TOOLS 权限过大、inbox 被提前消费等问题。
4	怎么实现？	实现 TEAM_TOOLS、修正 run_check_inbox、删除主程序直接 read_inbox、加入 has_inbox + inbox_processor_loop。
5	为什么 cache hit 连续打印？	不是错误；因为 get_system_prompt 被多处重复调用，而 context 内容常常没有变化，所以本地缓存连续命中。

一句话复盘：你已经不是在写“一个会调用工具的 agent”，而是在写“一个有调度中心、工作线程、消息队列、后台任务和安全边界的 agent runtime”。

1. 总体架构思维导图

s15 Agent Team 总体思维导图



- 四条底层原则：**
- 异步结果不补原 tool_use；最终结果走 MessageBus
 - 只有 Lead 能 spawn teammate；teammate 不递归派生
 - tool_use_id / task_id / hg_id / message_id 必须分离

图中最重要的边界是：Lead 是唯一调度中心；teammate 是后台 worker；MessageBus 不是 tool_result 协议，而是跨线程通信层。

2. s15 架构的正确心智模型

s15 的目标不是简单增加几个工具，而是让主循环具备“团队协作”的能力。Lead 负责判断任务如何拆分、何时派发、何时等待、何时汇总；teammate 只负责在自己限定权限内完成一个子任务，并把 summary 发回 Lead。

【重点】异步 teammate 完成任务时，不应该把最终结果硬插回原来的 tool_use 后面。原 tool_result 只表示“teammate 已启动”；真正结果应通过 MessageBus 回到 lead inbox，然后由 Lead 调用 check_inbox 读取。

组件	职责	不应该做的事
Lead Agent	维护 messages，调用 LLM，派发工具，spawn teammate，汇总结果	不应该被 teammate 直接修改 messages
Teammate Thread	后台执行指定子任务，最多若干轮，最终 send_message 给 lead	不应该再 spawn teammate，不应该直接向用户提问
MessageBus	用 .mailbox/*.jsonl 做跨线程消息队列	不应该混用 tool_use_id，也不应该替代 API tool_result 协议
check_inbox	Lead 消费 lead inbox 的唯一工具入口	主程序不要提前 BUS.read_inbox("lead")
cron/background	外部事件源，唤醒或注入 scheduled/background 通知	不应该绕过 agent_lock 并发修改 session_history

3. subagent 是否还需要？

答案是：需要，但它的位置要下降。以前 s06 的 subagent 是 Lead 直接调用的一次性子 agent；到了 s15，teammate 已经承担了“长期身份 + mailbox + 后台线程”的高层概念，所以 subagent 更适合作为 teammate 内部的执行内核。

概念	生命周期	上下文	通信方式	推荐定位
subagent	一次性	fresh messages[]	返回 summary	内部函数 run_subagent_once
teammate	长驻或后台线程	自己的 messages + mailbox	MessageBus 收发	对 Lead 暴露的团队能力

□□□□

```
Lead Loop
├─ spawn_teammate(name, role, prompt)
│   └─ Teammate Thread
│       └─ run_subagent_once(task, max_turns=10)
│           └─ summary -> BUS.send(name, "lead", summary, "result")
```

4. 异步结果回流：最容易混乱的一段

异步 teammate 结果回流流程



这里最容易犯错的是：主程序看到 `lead.jsonl` 后直接 `BUS.read_inbox("lead")`。因为你的 `MessageBus.read_inbox` 是 destructive read，会读完后 `unlink` 文件；这样 `check_inbox` 工具后续就读不到了。正确做法是：主程序只检测并唤醒 Lead，真正读取动作交给 `check_inbox`。

5. 代码审查：本次发现的关键问题

代码诊断：问题 -> 后果 -> 修复

问题	后果	修复
TOOLS 未暴露 TEAM_TOOLS	Lead 看到 system 说有工具，但 API schema 没有，模型不能合法调用	定义 TEAM_TOOLS，并 <code>TOOLS.extend(Team_TOOLS)</code>
SUB_TOOLS 放了 spawn_teammate	子 agent/teammate 可能递归 spawn，调度树失控	teammate 只保留 <code>bash/read/write/edit/glob/send_message</code>
teammate schema 用 path	<code>handler(**block.input)</code> 会调用 <code>run_read(path=...)</code> ，形参不匹配	统一改成 <code>file_path</code> ，或 <code>lambda path: run_read(path)</code>
teammate 绕过 hooks	后台线程可绕过权限检查，异常会让 <code>active_teammates</code> 残留	执行工具时加 <code>PreToolUse/PostToolUse</code> 、 <code>try/except/finally</code>
主程序直接 <code>read_inbox</code>	<code>lead.jsonl</code> 被提前 <code>unlink</code> ， <code>check_inbox</code> 后续读不到	改为 <code>has_inbox</code> 检测 + <code>inbox_notification</code> 唤醒 Lead
cache hit 连续打印	不是错误，只是重复刷新 context 且内容未变	减少重复 <code>get_system_prompt</code> ，或只在 debug 模式打印

按优先级，先修“Lead 工具 schema 未暴露”和“path/file_path 不匹配”。前者会导致模型根本不能合法调用 `spawn_teammate`；后者会导致 teammate 一调用 `read_file/write_file` 就 `TypeError`。

6. 具体实现方案

6.1 把 team tools 暴露给 Lead

模型能否调用工具，取决于传给 API 的 tools schema 列表，而不仅是 TOOL_HANDLERS。你需要把 spawn_teammate、send_message、check_inbox 的 schema 加进 TOOLS。

```
TEAM_TOOLS = [
    {
        "name": "spawn_teammate",
        "description": "Spawn a teammate in a background thread. Result is async; use check_inbox later.",
        "input_schema": {
            "type": "object",
            "properties": {
                "name": {"type": "string"},
                "role": {"type": "string"},
                "prompt": {"type": "string"},
            },
            "required": ["name", "role", "prompt"],
        },
    },
    {
        "name": "send_message",
        "description": "Send a message to a teammate via MessageBus.",
        "input_schema": {
            "type": "object",
            "properties": {"to": {"type": "string"}, "content": {"type": "string"}},
            "required": ["to", "content"],
        },
    },
    {
        "name": "check_inbox",
        "description": "Check Lead's inbox for teammate messages.",
        "input_schema": {"type": "object", "properties": {}, "required": []},
    },
]

TOOLS.extend(TOOL_HANDLERS)
```

6.2 让 check_inbox 返回结构化内容

```
def run_check_inbox() -> str:
    msgs = BUS.read_inbox("lead")
    if not msgs:
        return "(inbox empty)"

    text = json.dumps(msgs, ensure_ascii=False, indent=2)
    if len(text) > 12000:
        text = text[:12000] + "
... [truncated]"
    return f"<lead_inbox>
{text}
</lead_inbox>"
```

6.3 主程序不要直接消费 lead inbox

删除 `__main__` 中 `run_agent_turn_locked(query)` 后面的 `BUS.read_inbox("lead")`。它会把消息提前删掉。

```
# Before: run_agent_turn_locked(query), then BUS.read_inbox("lead")
# After:
with agent_lock:
    run_agent_turn_locked(query)
    print()
```

6.4 加检测型 `has_inbox` 和 `inbox_processor_loop`

这个 loop 只检测 `lead.jsonl` 是否存在，不读取内容。它通过一条 `inbox_notification` 唤醒 Lead，让 Lead 自己决定是否调用 `check_inbox`。

```
def has_inbox(agent: str) -> bool:
    inbox = MAILBOX_DIR / f"{agent}.jsonl"
    return inbox.exists() and inbox.stat().st_size > 0

def inbox_processor_loop():
    last_seen_mtime = 0.0
    while True:
        time.sleep(0.5)
        inbox = MAILBOX_DIR / "lead.jsonl"
        if not inbox.exists():
            continue
        try:
            mtime = inbox.stat().st_mtime
        except FileNotFoundError:
            continue
        if mtime == last_seen_mtime:
            continue
        if not agent_lock.acquire(blocking=False):
            continue
        try:
            if not has_inbox("lead"):
                continue
            last_seen_mtime = mtime
            run_agent_turn_locked(
                "<inbox_notification>"
                "Teammate messages may be available. Decide whether to call check_inbox."
                "</inbox_notification>"
            )
        finally:
            agent_lock.release()
```

6.5 在 `start_runtime()` 启动 `inbox processor`

```
def start_runtime():
    load_durable_jobs()
    threading.Thread(target=cron_scheduler_loop, daemon=True, name="cron-scheduler").start()
    threading.Thread(target=queue_processor_loop, daemon=True, name="cron-queue-processor").start()
    threading.Thread(target=inbox_processor_loop, daemon=True, name="lead-inbox-processor").start()
```

6.6 修正 teammate 局部工具 schema

把 teammate 的 `read_file/write_file` 参数从 `path` 改成 `file_path`，和 `run_read(file_path)`、`run_write(file_path, content)` 对齐。

```
# In teammate sub_tools:
{"name": "read_file", "input_schema": {
```

```

    "type": "object",
    "properties": {"file_path": {"type": "string"}},
    "required": ["file_path"]
  }}

{"name": "write_file", "input_schema": {
  "type": "object",
  "properties": {"file_path": {"type": "string"}, "content": {"type": "string"}},
  "required": ["file_path", "content"]
}}

```

7. 权限、安全与线程稳定性

teammate 执行工具时也要经过安全层。教学版可以先简化，但至少要有 try/except，避免线程异常退出后 active_teachmates 没有清理。更稳的版本应该给 teammate 工具执行也加 PreToolUse/PostToolUse。

```

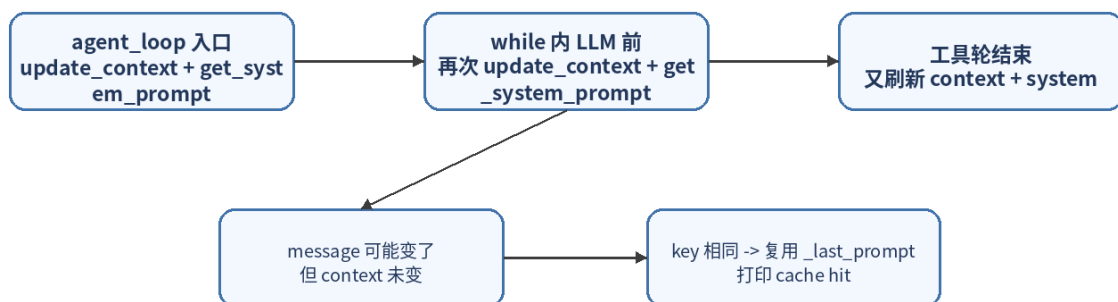
try:
    blocked = trigger_hooks("PreToolUse", block)
    if blocked:
        output = str(blocked)
    else:
        handler = sub_handlers.get(block.name)
        output = handler(**block.input) if handler else f"Unknown tool: {block.name}"
        trigger_hooks("PostToolUse", block, output)
except Exception as e:
    output = f"Error: {type(e).__name__}: {e}"
finally:
    # Better cleanup in the outer finally of teammate run()
    pass

```

【重点】权限模型上，Lead 可以 spawn_teachmate；teammate 可以 send_message；teammate 不应该 spawn_teachmate、schedule_cron、直接修改 Lead messages 或直接问用户。

8. [cache hit] 连续打印的原因

为什么 [cache hit] 会连续打印？



结论：它是本地 prompt 组装缓存日志，不等于 API 请求被跳过，也不是错误。

你的 get_system_prompt(context) 是本地缓存：把 context JSON 序列化成 key，如果 key 和上次一样，就复用 _last_prompt 并打印 cache hit。由于 update_context 返回的主要是 enabled_tools、workspace、skills、memory_index、memories，而不是完整 messages，所以 message 变了不一定导致 context 变。

它连续打印两三次，通常是因为 agent_loop 入口、while 内 LLM 调用前、工具轮结束后都重复调用了 get_system_prompt；这些调用之间 context 没变化，因此连续命中缓存。

这不是 API prompt cache，也不代表 LLM 请求没发出。它只是说明 Python 没有重新 assemble system prompt 字符串。

9. 最小可跑版本的修改顺序

1. 先定义 TEAM_TOOLS，并执行 TOOLS.extend(TEAM_TOOLS)。
2. 从 SUB_TOOLS 或 teammate 局部 tools 中移除 spawn_teammate/check_inbox，避免递归派生。
3. 把 teammate 局部 read_file/write_file 的 path 统一改成 file_path。
4. 把 run_check_inbox 改成结构化返回 JSON。
5. 删除 __main__ 里直接 BUS.read_inbox("lead") 的代码。
6. 新增 has_inbox + inbox_processor_loop，只提醒 Lead，不消费消息。
7. 在 start_runtime() 中启动 inbox_processor_loop。
8. 给 teammate 工具执行加 try/except/finally，防止 active_teammates 残留。
9. 把 system prompt 加上 team_policy，提醒模型 spawn_teammate 是异步结果。

10. 测试清单

测试项	预期结果
打印 [t["name"] for t in TOOLS]	能看到 spawn_teammate、send_message、check_inbox
让 Lead spawn alice	立即返回 spawned，不等待 alice 完整执行
alice 调 read_file	不会出现 run_read() got unexpected keyword argument path
alice 完成后 .mailbox/lead.jsonl 存在	主程序不删除文件，只注入 inbox_notification
Lead 调用 check_inbox	返回 <lead_inbox>...</lead_inbox>，随后 lead.jsonl 被消费
再次 check_inbox	返回 (inbox empty)
cache hit 连续打印	理解为正常本地缓存命中，不当成 bug

11. 最终记忆点

- TOOLS 是模型可调用工具的 schema；TOOL_HANDLERS 只是 Python 执行映射。两者必须同时有。
- tool_use_id 只服务于 API 的 tool_use/tool_result 配对；不要拿它当 task_id、bg_id、message_id。
- teammate 的最终结果属于异步事件，不应该伪装成原 tool_use 的同步 tool_result。
- MessageBus.read_inbox 是 destructive read；因此只有 check_inbox 应该消费 lead inbox。
- subagent 在 s15 中应是内部执行内核，不应继续作为和 teammate 同级的高层能力。
- cache hit 连续打印说明 context 未变，不说明程序卡住。

一句话总结：s15 的关键不是“多线程本身”，而是“Lead 主控权 + teammate 有边界执行 + MessageBus 异步回流 + check_inbox 消费结果”这套协议是否清晰。

附录：你给出的 s15 架构草图

下面是本次讨论开始时的原始架构图，PDF 前文已经将其重新整理成文字化架构和思维导图。

